Self-Paced Module: Java File Handling

Java Programming · Grade 12 TVL · Cielito Zamora Senior High School · Week 15, October 26–30, 2026 · Continues the Collections Framework module

Name: _____
Section: _____ Date: _____

Day	Time	Topic	What you will do
Mon, Oct 26	2 hours	The File class: create, check, and delete	Pages 1–2, Steps 1–2. Answer the **Monday strip** (page 7, bottom).
Tue, Oct 27	2 hours	Reading: FileReader and BufferedReader	Pages 2–3, Steps 3–4. Answer the **Tuesday strip** (page 7, middle).
Wed, Oct 28	2 hours	Reading: Scanner and StringReader	Pages 3–4, Steps 5–6. Answer the **Wednesday strip** (page 7, top).
Thu, Oct 29	3 hours	Writing: FileWriter, BufferedWriter, StringWriter; append vs. overwrite	Pages 4–5, Steps 7–10. Answer the **Thursday strip** (page 8, bottom). Start your Performance Task.
Fri, Oct 30	3 hours	**PERFORMANCE TASK**	Page 6. Finish, present, and answer questions. Answer the **Friday strip** (page 8, top).

How to use this module: Work in order. Type every sample code and run it — do not just read it. Each day ends with a **LIPLET**, a short written quiz on **page 7 or 8**. Tear off **only** that day's strip, so you keep the rest of the module until Friday.

No internet needed. Everything you need is in this module and on your computer with Java (JDK) installed. Keep every class in its own file, all in one folder. Compile with `javac *.java`, run with `java Main`. The text files you make this week (`fleet.txt`, `fares.txt`, and the rest) go in **the same folder**. Stuck on an error? See the **Error Guide on page 5**.

WHERE WE LEFT OFF — your Vehicle program from the Collections module

Your folder should already have: `Vehicle.java`, `Jeepney.java`, `Tricycle.java`, `Bus.java`, `OverloadException.java`, `InvalidDistanceException.java`, and `VehicleNotFoundException.java`. None of them change this week. You will replace `Main.java` many times, so save a copy first as `Main_week14.txt`.

The problem this week fixes: last week your fleet lived in an `ArrayList`. When the program ended, the list disappeared. Run it tomorrow and Mang Ramon's jeepney is gone again. A program that must **remember** needs to save its data in a **file**.

PART A — WHAT IS FILE HANDLING?

Definition: File handling means a program creating, reading, writing, and deleting files on the computer's disk, so data stays after the program ends. Java's tools for this are in the `java.io` package (io = input and output). A **text file** holds plain characters you can open in Notepad.

ANALOGY The barangay hall keeps records in a **filing cabinet**. Writing a label "Fleet records" on an index card does **not** put a folder in the cabinet — that is a `File` object. Someone still has to make the folder (**create**), look inside it (**read**), add pages (**write**), or throw it out (**delete**). The cabinet keeps the folder even after the clerk goes home — just like a file stays after your program ends.

Day	Java class	Its job
Mon	`File`	names a file or folder; creates, checks, lists, and deletes it
Tue	`FileReader`, `BufferedReader`	read a text file one character, or one whole line, at a time
Wed	`Scanner`, `StringReader`	read words and numbers from a file; read a String as if it were a file
Thu	`FileWriter`, `BufferedWriter`, `StringWriter`	write to a file (replace it, or add to the end); build text in memory

MONDAY STEP 1 — A File object is only a name, until you create it

STEP 1 Replace `Main.java` with this, then run it **once**.

```
import java.io.File;
import java.io.IOException;

public class Main {
    public static void main(String[] args) {
        File report = new File("fleet.txt");   // only a NAME so far, no file yet
        System.out.println("File name: " + report.getName());
        System.out.println("Exists before? " + report.exists());
        try {
            boolean made = report.createNewFile();   // now the real file is made
            System.out.println("Created now? " + made);
        } catch (IOException e) {
            System.out.println("Could not create: " + e.getMessage());
        }
        System.out.println("Exists after? " + report.exists());
        System.out.println("Is a file? " + report.isFile());
        System.out.println("Size in bytes: " + report.length());
    }
}
```

Output (first run):

```
File name: fleet.txt
Exists before? false
Created now? true
Exists after? true
Is a file? true
Size in bytes: 0
```

Look closely: `new File("fleet.txt")` did not make a file — `exists()` said **false**. Only `createNewFile()` made the real (empty) file, so its size is **0 bytes**. Open your folder: `fleet.txt` is there now. Keep it — you will fill it on Tuesday.

Check yourself: run Step 1 a **second** time without changing anything. Predict first: what will `Exists before?` and `Created now?` say? (The file is already there, so `createNewFile()` refuses and returns **false**. It does not erase the file.) Also: `createNewFile()` can throw an `IOException`, a **checked** exception from Week 13, which is why it sits inside `try`.

File methods you will use this week (Step 2 tries the rest)

File method	What it does	Gives back
`exists()`	checks if the file or folder is really on the disk	true / false
`createNewFile()`	makes a new, empty file (only if it is not there yet)	true if made; false if it already existed
`mkdir()`	makes a new folder	true if made
`isFile()`, `isDirectory()`	is it a file? is it a folder?	true / false
`length()`, `getName()`	size in bytes; the name only	a number; a String
`list()`	the names of everything inside a folder	a String array
`delete()`	deletes a file, or an **empty** folder	true if deleted

Words to know this week. Path: where a file is; a plain name like `fleet.txt` means "in the folder where you run `java Main`." **Read**: bring data from a file into the program. **Write**: send data from the program into a file. **Overwrite**: erase the file and write it fresh. **Append**: add to the end and keep what was there. **Close**: tell Java you are done with a file, so everything is saved and the file is released.

MONDAY STEP 2 — Folders, listing, and deleting

A `File` can also name a **folder** (also called a directory). `new File(folder, "trips.txt")` means "trips.txt inside that folder."

STEP 2 Replace `Main.java`:

```
import java.io.File;
import java.io.IOException;
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        File folder = new File("records");            // a folder is also a File
        System.out.println("Folder made? " + folder.mkdir());
        System.out.println("Is a folder? " + folder.isDirectory());

        File trips = new File(folder, "trips.txt");    // records/trips.txt
        File old = new File(folder, "old_log.txt");    // records/old_log.txt
        try {
            trips.createNewFile();
            old.createNewFile();
        } catch (IOException e) {
            System.out.println("Could not create: " + e.getMessage());
        }

        String[] names = folder.list();               // names inside the folder
        Arrays.sort(names);                           // list() promises no order
        System.out.println("Inside records: " + Arrays.toString(names));

        System.out.println("Delete old_log.txt? " + old.delete());
        System.out.println("Delete it again? " + old.delete());
        System.out.println("Delete the folder? " + folder.delete());
        System.out.println("Left inside: " + Arrays.toString(folder.list()));
    }
}
```

Output:

```
Folder made? true
Is a folder? true
Inside records: [old_log.txt, trips.txt]
Delete old_log.txt? true
Delete it again? false
Delete the folder? false
Left inside: [trips.txt]
```

Read it carefully: the second `delete()` gave **false** — there was nothing left to delete. The folder also refused to be deleted: Java only deletes a folder when it is **empty**, and `trips.txt` is still inside. These methods **return false** instead of throwing an exception, so always check what they return.

Check yourself: run Step 2 again. Now `Folder made?` says **false** (the folder already exists) and the list shows `old_log.txt` again, because `createNewFile()` made it again. Why does `Arrays.sort(names)` come before printing? (`list()` promises no order, so sorting makes everyone's output match.)

LIPLET · MONDAY Stop here and answer the **MONDAY strip (bottom of page 7)**, in pen. Tear it off and submit it.

TUESDAY PART B — READING A FILE: FileReader AND BufferedReader

Definition: A **reader** is an object that brings characters **from** a source into your program. `FileReader` reads a text file **one character at a time**. `BufferedReader` wraps around another reader and adds `readLine()`, which reads a **whole line** at once and gives `null` when there are no more lines.

ANALOGY The dispatcher reads the terminal logbook to the drivers. `FileReader` is like spelling it out letter by letter: "N... X... A..." — correct, but slow and tiring. `BufferedReader` is like reading one full line of the logbook at a time: "Jeepney, NXA 123, Mang Ramon." When she reaches the last line, there is nothing more to read: that is `null`.

STEP 3a Open `fleet.txt` (made in Step 1) in **Notepad**. Type these four lines **exactly**: commas, no spaces after commas. Save it.

```
Jeepney,NXA 123,Mang Ramon,Monumento-Divisoria
Tricycle,TRC 456,Kuya Ben,Bagong Silang TODA
Bus,BUS 789,Aling Rosa,50
Jeepney,NYB 321,Mang Tonyo,Sangandaan-Recto
```

New: try-with-resources. A file you open must be **closed** when you are done. If you write `try (FileReader reader = new FileReader(...)) { ... }`, Java closes it for you at the end, even if an exception happens.

STEP 3 Replace `Main.java`. This reads the file one character at a time.

```
import java.io.FileReader;
import java.io.IOException;

public class Main {
    public static void main(String[] args) {
        String firstLine = "";
        int commas = 0;
        boolean firstDone = false;
        try (FileReader reader = new FileReader("fleet.txt")) {  // closes by itself
            int ch = reader.read();          // ONE character, given as a number
            while (ch != -1) {               // -1 means "end of file"
                if (ch == '\n') {
                    firstDone = true;        // the first line has ended
                } else if (ch == ',') {
                    commas++;
                }
                if (!firstDone && ch != '\r') {
                    firstLine = firstLine + (char) ch;   // number back to a letter
                }
                ch = reader.read();
            }
        } catch (IOException e) {
            System.out.println("Cannot read: " + e.getMessage());
        }
        System.out.println("First line: " + firstLine);
        System.out.println("Commas in the file: " + commas);
    }
}
```

Output:

```
First line: Jeepney,NXA 123,Mang Ramon,Monumento-Divisoria
Commas in the file: 12
```

`read()` gives each character as a **number** (its code), and **-1** at the end of the file. That is a lot of work just to get one line.

TUESDAY STEP 4 — BufferedReader: load the whole fleet from fleet.txt

STEP 4 Replace `Main.java`. Each line of text becomes a real `Vehicle` object in last week's `ArrayList`.

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public class Main {
    // turns ONE line of text into a Vehicle object
    static Vehicle toVehicle(String line) {
        String[] part = line.split(",");   // "Bus,BUS 789,Aling Rosa,50" -> 4 parts
        if (part[0].equals("Jeepney")) {
            return new Jeepney(part[1], part[2], part[3]);
        } else if (part[0].equals("Tricycle")) {
            return new Tricycle(part[1], part[2], part[3]);
        } else {
            return new Bus(part[1], part[2], Integer.parseInt(part[3]));
        }
    }

    public static void main(String[] args) {
        List<Vehicle> fleet = new ArrayList<>();
        try (BufferedReader in = new BufferedReader(new FileReader("fleet.txt"))) {
            String line = in.readLine();       // a whole line, or null at the end
            while (line != null) {
                if (!line.trim().isEmpty()) {  // skip blank lines
                    fleet.add(toVehicle(line));
                }
                line = in.readLine();
            }
        } catch (IOException e) {
            System.out.println("Cannot read: " + e.getMessage());
        }
        System.out.println("Loaded " + fleet.size() + " vehicles from fleet.txt");
        for (Vehicle v : fleet) {
            v.showInfo();
        }
    }
}
```

Output:

```
Loaded 4 vehicles from fleet.txt
NXA 123 | Driver: Mang Ramon | Passengers: 0/20
TRC 456 | Driver: Kuya Ben | Passengers: 0/4
BUS 789 | Driver: Aling Rosa | Passengers: 0/50
NYB 321 | Driver: Mang Tonyo | Passengers: 0/20
```

	FileReader alone	BufferedReader (around a FileReader)
Reads	one character: `read()`	one whole line: `readLine()`
End of file	returns `-1`	returns `null`
Use it for	counting or checking single characters	data files with one record per line (most of the time)

Look closely: `toVehicle()` returns a `Jeepney`, a `Tricycle`, or a `Bus`, but its return type is `Vehicle`. That is **polymorphism** from the Four Pillars. `split(",")` cuts a line at every comma into an array; `Integer.parseInt("50")` turns the text "50" into the number 50.

Check yourself: (1) Change `"fleet.txt"` to `"flet.txt"` and run. You get `Cannot read: flet.txt (No such file or directory)` — on Windows it ends *(The system cannot find the file specified)*. Change it back. (2) Add the line `Bus,BUS 000,Mang Jun,fifty` to `fleet.txt` and run. It crashes with a `NumberFormatException`. Remove the line. You will learn to handle that in your Performance Task.

LIPLET · TUESDAY Stop here and answer the **TUESDAY strip (middle of page 7)**, in pen. Tear it off and submit it.

WEDNESDAY PART C — SCANNER ON A FILE, AND StringReader

Definition: You already know `Scanner` from reading the keyboard (`new Scanner(System.in)`). Give it a `File` instead — `new Scanner(new File("fares.txt"))` — and it reads the file word by word and **number by number**. `hasNextDouble()` looks ahead and answers "is the next word a number?" without using it up.

ANALOGY At the end of the day, the canteen cashier runs her finger along the price list and adds up only the **numbers**, skipping spaces and new lines. Before adding, she checks: "Is this a number?" If she hits a word like "abc," she stops instead of adding nonsense. That cashier is `Scanner` with `hasNextDouble()`.

STEP 5a In Notepad, make a new file `fares.txt` in the same folder, with these two lines (sample fares, not real LTFRB rates):

```
13.00 15.00 20.50
27.50 13.00
```

STEP 5 Replace `Main.java`:

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        double total = 0;
        int count = 0;
        try (Scanner sc = new Scanner(new File("fares.txt"))) {  // a FILE, not System.in
            while (sc.hasNextDouble()) {          // is the next word a number?
                total = total + sc.nextDouble();
                count++;
            }
        } catch (FileNotFoundException e) {
            System.out.println("Missing file: " + e.getMessage());
        }
        System.out.println("Fares read: " + count);
        System.out.println("Total collected: P" + total);
        if (count > 0) {
            System.out.printf("Average fare: P%.2f%n", total / count);
        }
    }
}
```

Output:

```
Fares read: 5
Total collected: P89.0
Average fare: P17.80
```

Check yourself: change `fares.txt` to `13.00 15.00 abc` on line 1 and run. Now `Fares read: 2`: `hasNextDouble()` said "not a number" at `abc`, so the loop stopped safely. If you call `nextDouble()` without asking first, it crashes with `InputMismatchException`. Put the file back.

WEDNESDAY STEP 6 — StringReader: a String that acts like a file

Definition: `StringReader` is a reader whose source is a **String** in memory, not a file on disk. Because `FileReader` and `StringReader` are **both** kinds of `Reader`, one method can read from either.

ANALOGY Before reading the real class record, a teacher practises on a **photocopy** of two sample rows. Same reading method, safer source. `StringReader` is that photocopy: you can test your reading code without touching the real file.

STEP 6 Replace `Main.java`:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.io.Reader;
import java.io.StringReader;

public class Main {
    // works with ANY Reader: a real file, or a String acting like a file
    static int countVehicles(Reader source) throws IOException {
        BufferedReader in = new BufferedReader(source);
        int count = 0;
        String line = in.readLine();
        while (line != null) {
            if (!line.trim().isEmpty()) {
                count++;
            }
            line = in.readLine();
        }
        in.close();
        return count;
    }

    public static void main(String[] args) {
        String sample = "Jeepney,NXA 123,Mang Ramon,Monumento-Divisoria\n"
                      + "Tricycle,TRC 456,Kuya Ben,Bagong Silang TODA\n";
        try {
            System.out.println("From a String: " + countVehicles(new StringReader(sample)));
            System.out.println("From fleet.txt: " + countVehicles(new FileReader("fleet.txt")));
            System.out.println("From an empty String: " + countVehicles(new StringReader("")));
        } catch (IOException e) {
            System.out.println("Cannot read: " + e.getMessage());
        }
    }
}
```

Output:

```
From a String: 2
From fleet.txt: 4
From an empty String: 0
```

Look closely: `countVehicles(Reader source)` accepts a `StringReader` **and** a `FileReader`. The parameter type is the parent class `Reader`: the same idea as a `Vehicle` variable holding a `Jeepney`.

	BufferedReader	Scanner	StringReader
Reads from	any Reader (file, String)	keyboard, file, or String	a String in memory
Best at	whole lines: one record per line	words and numbers: `nextInt()`, `nextDouble()`	testing reading code without a real file

LIPLET · WEDNESDAY Stop here and answer the **WEDNESDAY strip (top of page 7)**, in pen. Tear it off and submit it.

THURSDAY PART D — WRITING FILES: OVERWRITE OR APPEND?

Definition: A **writer** sends characters **from** your program **to** a destination. `new FileWriter("trip_log.txt")` **overwrites**: it erases the file and starts it fresh. `new FileWriter("trip_log.txt", true)` **appends**: it keeps what is there and adds to the end. If the file does not exist yet, both make it.

ANALOGY The terminal has a **whiteboard** for "Next jeepney to leave" — each new note erases the old one (**overwrite**). It also has a **logbook** where every departure is a new line under the last one (**append**). Choose the wrong one and either the logbook loses its history or the whiteboard fills up with old news.

STEP 7 Replace `Main.java`. `showFile()` prints a file so you can see inside it; you will reuse it.

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class Main {
    // prints every line of a file, so we can SEE what is inside
    static void showFile(String name) {
        System.out.println("--- " + name + " ---");
        try (BufferedReader in = new BufferedReader(new FileReader(name))) {
            String line = in.readLine();
            while (line != null) {
                System.out.println(line);
                line = in.readLine();
            }
        } catch (IOException e) {
            System.out.println("Cannot read: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        try (FileWriter out = new FileWriter("trip_log.txt")) {   // OVERWRITE mode
            out.write("NXA 123 left Monumento\n");
        } catch (IOException e) {
            System.out.println("Cannot write: " + e.getMessage());
        }
        try (FileWriter out = new FileWriter("trip_log.txt")) {   // same file again
            out.write("TRC 456 left Bagong Silang\n");
        } catch (IOException e) {
            System.out.println("Cannot write: " + e.getMessage());
        }
        showFile("trip_log.txt");
    }
}
```

Output:

```
--- trip_log.txt ---
TRC 456 left Bagong Silang
```

Only one line! The second `FileWriter` erased the NXA 123 line. That is overwrite mode.

THURSDAY STEP 8 — Append mode: the logbook grows

STEP 8 Keep `showFile()` from Step 7. Replace only the `main` method with this:

```
public static void main(String[] args) {
    try (FileWriter out = new FileWriter("trip_log.txt", true)) {   // true = APPEND
        out.write("BUS 789 left Caloocan\n");
        out.write("NYB 321 left Sangandaan\n");
    } catch (IOException e) {
        System.out.println("Cannot write: " + e.getMessage());
    }
    showFile("trip_log.txt");
}
```

Output:

```
--- trip_log.txt ---
TRC 456 left Bagong Silang
BUS 789 left Caloocan
NYB 321 left Sangandaan
```

Check yourself: run Step 8 again. Predict: how many lines now? (Five: append never erases, so the two new lines are added again.)

THURSDAY STEP 9 — BufferedWriter: save the fleet, then load it back

Definition: `BufferedWriter` wraps around a `FileWriter` and collects text in memory, then writes it in big pieces — faster for many lines. `newLine()` ends a line the right way for each computer (Windows and Linux end lines differently). Its text reaches the file only when the writer is **closed**, so always use try-with-resources.

STEP 9 Replace `Main.java`. `toLine()` is the **reverse** of `toVehicle()`: it turns an object back into one line of text.

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public class Main {
    static Vehicle toVehicle(String line) {           // same as Step 4
        String[] part = line.split(",");
        if (part[0].equals("Jeepney")) {
            return new Jeepney(part[1], part[2], part[3]);
        } else if (part[0].equals("Tricycle")) {
            return new Tricycle(part[1], part[2], part[3]);
        } else {
            return new Bus(part[1], part[2], Integer.parseInt(part[3]));
        }
    }

    // the REVERSE of toVehicle: one Vehicle back into one line of text
    static String toLine(Vehicle v) {
        String middle = "," + v.getPlateNumber() + "," + v.getDriverName() + ",";
        if (v instanceof Jeepney) {
            return "Jeepney" + middle + ((Jeepney) v).getRoute();
        } else if (v instanceof Tricycle) {
            return "Tricycle" + middle + ((Tricycle) v).getToda();
        } else {
            return "Bus" + middle + ((Bus) v).getSeats();
        }
    }

    static List<Vehicle> load(String fileName) {
        List<Vehicle> fleet = new ArrayList<>();
        try (BufferedReader in = new BufferedReader(new FileReader(fileName))) {
            String line = in.readLine();
            while (line != null) {
                if (!line.trim().isEmpty()) {
                    fleet.add(toVehicle(line));
                }
                line = in.readLine();
            }
        } catch (IOException e) {
            System.out.println("Cannot read " + fileName + ": " + e.getMessage());
        }
        return fleet;
    }

    public static void main(String[] args) {
        List<Vehicle> fleet = load("fleet.txt");
        System.out.println("Loaded from fleet.txt: " + fleet.size());
        fleet.add(new Tricycle("TRC 654", "Mang Lito", "Bagong Silang TODA"));

        String backup = "fleet_backup.txt";
        try (BufferedWriter out = new BufferedWriter(new FileWriter(backup))) {
            for (Vehicle v : fleet) {
                out.write(toLine(v));
                out.newLine();            // ends the line
            }
        } catch (IOException e) {
            System.out.println("Cannot write: " + e.getMessage());
        }
        System.out.println("Saved to " + backup + ": " + fleet.size());

        List<Vehicle> again = load(backup);
        System.out.println("Reloaded: " + again.size());
        again.get(again.size() - 1).showInfo();
    }
}
```

Output:

```
Loaded from fleet.txt: 4
Saved to fleet_backup.txt: 5
Reloaded: 5
TRC 654 | Driver: Mang Lito | Passengers: 0/4
```

Open `fleet_backup.txt` in Notepad: five lines, the new tricycle last. Close the program, run it tomorrow — the data is still there.

THURSDAY STEP 10 — StringWriter: write a report in memory first

Definition: `StringWriter` is a writer whose destination is a **String in memory**, not a file. You write into it piece by piece, then `toString()` gives you all the text at once — to show on screen, save to a file, or both.

ANALOGY The dispatcher drafts the daily report on **scratch paper** first, then reads it aloud and copies the same text into the record book. `StringWriter` is the scratch paper.

STEP 10 Replace `Main.java`. It reads the backup from Step 9.

```
import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.io.StringWriter;

public class Main {
    public static void main(String[] args) {
        StringWriter report = new StringWriter();   // a "file" that lives in memory
        report.write("DAILY FLEET REPORT\n");
        int count = 0;
        try (BufferedReader in = new BufferedReader(new FileReader("fleet_backup.txt"))) {
            String line = in.readLine();
            while (line != null) {
                String[] part = line.split(",");
                count++;
                report.write(count + ". " + part[1] + " (" + part[0] + ") - " + part[2] + "\n");
                line = in.readLine();
            }
        } catch (IOException e) {
            System.out.println("Cannot read: " + e.getMessage());
        }
        report.write("Total vehicles: " + count + "\n");

        String text = report.toString();     // everything written so far
        System.out.print(text);              // 1) show it on screen
        try (FileWriter out = new FileWriter("report.txt")) {
            out.write(text);                 // 2) save the SAME text
        } catch (IOException e) {
            System.out.println("Cannot write: " + e.getMessage());
        }
        File saved = new File("report.txt");
        System.out.println("report.txt size: " + saved.length() + " bytes");
    }
}
```

Output:

```
DAILY FLEET REPORT
1. NXA 123 (Jeepney) - Mang Ramon
2. TRC 456 (Tricycle) - Kuya Ben
3. BUS 789 (Bus) - Aling Rosa
4. NYB 321 (Jeepney) - Mang Tonyo
5. TRC 654 (Tricycle) - Mang Lito
Total vehicles: 5
report.txt size: 202 bytes
```

Check yourself: open `report.txt` in Notepad. It holds exactly the lines you saw on screen, because both came from the same `text`. Add one more line with `report.write("Prepared by: (your name)\n");` before `toString()` and run again. Where does your name appear?

Check your folder after Step 10	
Classes	`Main.java` + the 7 class files from last week
Typed by you	`fleet.txt`, `fares.txt`
Made by your programs	`records/trips.txt`, `trip_log.txt`, `fleet_backup.txt`, `report.txt`

LIPLET · THURSDAY Stop here and answer the **THURSDAY strip (bottom of page 8)**, in pen. Tear it off, submit it, then start the Performance Task on page 6.

ERROR GUIDE — no internet? Look up your error here

If you see...	It means...	Fix it by...
`unreported exception IOException; must be caught or declared to be thrown`	`createNewFile()` or a writer can throw a **checked** exception.	Put the line inside `try { } catch (IOException e) { }`.
`unreported exception FileNotFoundException; must be caught or declared to be thrown`	`new FileReader(...)` or `new Scanner(new File(...))` outside a try.	Use try-with-resources with a `catch`.
`cannot find symbol … symbol: class File`	A missing import.	Add `import java.io.File;` (or the class named).
`fleet.txt (No such file or directory)` Windows: *(The system cannot find the file specified)*	Wrong file name, or the file is in another folder.	Check the spelling and put the .txt file next to `Main.java`.
`NumberFormatException: For input string: "fifty"`	`Integer.parseInt()` got text that is not a number.	Fix the line, or catch `NumberFormatException` and skip it.
`ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3`	A line has fewer commas (parts) than `part[3]` needs.	Make every line complete, e.g. `Jeepney,NXA 999,Mang Jun,Recto`.
`java.util.InputMismatchException`	`nextDouble()` met a word that is not a number.	Ask `hasNextDouble()` first (Step 5).
No error, but the file is **empty** (0 bytes)	The writer was never closed, so its text never reached the disk.	Open every writer in try-with-resources.

THU-FRI PART E — PERFORMANCE TASK: MAKE YOUR PROGRAM REMEMBER

Open your own program from last week's Performance Task (your sari-sari store, pets, gadgets, school personnel, or shapes theme). Make it **save** its data to a text file and **load** it back the next time it runs. Start on Thursday after the Thursday strip; finish and present on Friday.

Requirements checklist — tick each box when done:

- ☐ A data file with one object per line (at least 4 lines, at least 2 subclasses)
- ☐ `exists()` checks the data file first; if it is missing, say so politely
- ☐ Load with `BufferedReader` + `readLine()` into your List
- ☐ A `toObject` / `toLine` pair: text → object, object → text
- ☐ Save the List back with `BufferedWriter` and `newLine()` (overwrite)
- ☐ Append one line to a log file each run: `new FileWriter(name, true)`
- ☐ Build a short report with `StringWriter`; show it and save it
- ☐ A bad line is **skipped** with a message (`NumberFormatException`), never a crash
- ☐ Every reader and writer is inside try-with-resources
- ☐ Make a temporary file, then `delete()` it and print the result
- ☐ A comment on each part, e.g. `// APPEND: keep old log lines`

Reuse this week's code. Your `toObject` works like `toVehicle()` (Step 4). Your `toLine` works like `toLine()` (Step 9). Use `showFile()` (Step 7) to check what your files hold.
A good order for main: 1) check and load the data file → 2) make the changes → 3) append to the log → 4) save the data file → 5) build and save the report.

Example for one theme (sari-sari store). Do yours your own way. `stock.txt` before the first run:

```
Bread,10
Softdrink,24
Rice (kg),fifty
```

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Map;
import java.util.TreeMap;

public class Main {
    public static void main(String[] args) {
        Map<String, Integer> stock = new TreeMap<>();
        String file = "stock.txt";
        try (BufferedReader in = new BufferedReader(new FileReader(file))) {
            String line = in.readLine();
            while (line != null) {
                String[] p = line.split(",");
                try {
                    stock.put(p[0], Integer.parseInt(p[1]));
                } catch (NumberFormatException e) {
                    System.out.println("Skipped bad line: " + line);
                }
                line = in.readLine();
            }
        } catch (IOException e) {
            System.out.println("No stock yet: " + e.getMessage());
        }
        // Ana buys 2 Bread
        stock.put("Bread", stock.getOrDefault("Bread", 0) - 2);
        // APPEND: the sales log keeps every old line
        try (FileWriter log = new FileWriter("sales_log.txt", true)) {
            log.write("Ana bought 2 Bread\n");
        } catch (IOException e) {
            System.out.println("Log failed: " + e.getMessage());
        }
        // OVERWRITE: stock.txt always holds today's numbers
        try (BufferedWriter out = new BufferedWriter(new FileWriter(file))) {
            for (Map.Entry<String, Integer> e : stock.entrySet()) {
                out.write(e.getKey() + "," + e.getValue());
                out.newLine();
            }
        } catch (IOException e) {
            System.out.println("Save failed: " + e.getMessage());
        }
        System.out.println("Stock saved: " + stock);
    }
}
```

Output (first run):

```
Skipped bad line: Rice (kg),fifty
Stock saved: {Bread=8, Softdrink=24}
```

Day	First half (about 1.5 hours)	Second half (about 1.5 hours)	Before you leave
Thursday	Steps 7 to 10 on pages 4–5; Thursday strip	Fill in the plan below; write your data file and your load part	Your program loads your data file and shows it
Friday	Save, log, report, bad-line handling; a classmate tries to break it	Presentations and Q&A	Friday strip handed in

Testing tip: before you present, run your program at least **four** ways: (1) normally; (2) a second time — the log must grow, the data must not double; (3) with the data file renamed — a polite "no data" message; (4) with a bad line added — that line is skipped. Every run must end normally, never with an `Exception in thread "main"` report.

Plan before you type — fill this in, and show it to your teacher:

File	File name	One line looks like...	Overwrite or append? Why?
Data file			
Log file			
Report file			

Starter template for the load part of your `Main.java`:

```
List<_____> items = new ArrayList<>();
try (BufferedReader in = new BufferedReader(new FileReader("_____.txt"))) {
    String line = in.readLine();
    while (line != null) {
        try {
            items.add(to_____(line));   // text -> object
        } catch (NumberFormatException e) {
            System.out.println("Skipped bad line: " + line);
        }
        line = in.readLine();
    }
} catch (IOException e) {
    System.out.println("No data yet: " + e.getMessage());
}
```

Self-check before you present — tick only what is honestly true:

I can explain, using my own code...	Yes	Not yet
why my data file overwrites but my log appends	☐	☐
what happens when my data file is missing	☐	☐
why every writer must be closed	☐	☐
how one line becomes one object, and back	☐	☐

Friday: your 5-minute presentation, in this order

1. **Introduce** your theme and your files: data, log, report. *(30 sec)*
2. **Run it:** show the data loading and the report. *(1 min)*
3. **Prove it live:** run again and open the log in Notepad; rename the data file; add a bad line. *(2 min)*
4. **Point to the code** for load, save, and append, and say why. *(1 min)*
5. **Answer questions.** *(30 sec)*

Prepare for these questions:

- Why overwrite for the data file but append for the log?
- What does your program do if the data file is missing?
- What does try-with-resources do for you?
- When would Scanner be better than BufferedReader?
- Why does your toObject method return the parent class?

Scoring Rubric (50 points)

Criterion	What earns full points	Points	Score
Load	At least 4 objects from 2+ subclasses load from the data file with BufferedReader; text becomes objects.	8	
Save	The List is saved back with BufferedWriter and newLine(); running twice does not double the data.	8	
Log + report	The log grows by appending; a StringWriter report is shown and saved.	8	
File class	exists() is checked before loading; a temporary file is created and deleted.	6	
Error handling	Missing file and bad line give friendly messages; try-with-resources everywhere; no crash.	10	
Presentation and Q&A	Proves each part live; explains overwrite vs. append and every choice with own code.	10	
	TOTAL	**50**	

LIPLET · FRIDAY After your presentation, answer the **FRIDAY strip (top of page 8)**, in pen, and hand in that sheet.

WEDNESDAY · OCT 28 Reading: Scanner and StringReader

Name: _____
Section: _____ Score: ____ / 15

Answer after Step 6 on page 4. This is the last strip on this sheet — hand in what is left.

3.1 Define (5 pts): How is `new Scanner(new File("fares.txt"))` different from `new Scanner(System.in)`? What does `hasNextDouble()` do?

3.2 Explain (5 pts): In Step 5's Check yourself, a file with `abc` in it gave `Fares read: 2` instead of crashing. Why? What would `nextDouble()` alone have done?

3.3 Connect (5 pts): In Step 6, `countVehicles(Reader source)` accepted both a `StringReader` and a `FileReader`. Which OOP pillar from Week 12 makes this work? Why is a StringReader useful for testing?

✂ CUT ON TUESDAY · DETACH THE TUESDAY STRIP BELOW THIS LINE ✂

TUESDAY · OCT 27 Reading: FileReader and BufferedReader

Name: _____
Section: _____ Score: ____ / 20

Answer after Step 4 on page 3.

2.1 Define (5 pts): What does `readLine()` give back, and what does it give back when the file has no more lines?

2.2 Explain (5 pts): Why do we open the reader inside `try ( ... )` (try-with-resources)? What does Java do for us at the end?

2.3 Identify (5 pts): Write the class or method that does each job.
a) reads one character _____ b) reads one whole line _____
c) turns "50" into 50 _____ d) cuts a line at every comma _____

2.4 Connect (5 pts): In Step 4, `toVehicle()` returns a Jeepney, Tricycle, or Bus, but its return type is `Vehicle`. Which Week 12 idea is this, and why is it useful here?

✂ CUT ON MONDAY · DETACH THE MONDAY STRIP BELOW THIS LINE ✂

MONDAY · OCT 26 The File class: create, check, and delete

Name: _____
Section: _____ Score: ____ / 15

Answer after Step 2 on page 2.

1.1 Define (5 pts): What is a `File` object? Does `new File("fleet.txt")` create the file on the disk? How do you know (Step 1)?

1.2 Explain (5 pts): In Step 1, `createNewFile()` gave **true** on the first run and **false** on the second. Why?

1.3 Connect (5 pts): In Step 2, `folder.delete()` gave **false**. Why? How is “returning false” different from throwing an exception like Week 13's `OverloadException`?

FRIDAY · OCT 30 Performance Task Exit Questions

Name: _____
Section: _____ Score: _____ / 20

Answer after your presentation, about your own program. Hand in this whole sheet.

5.1 Point (5 pts): Name your data file and your log file. Write what one line of your data file looks like, and the method that turns it into an object.

5.2 Evaluate (5 pts): You used overwrite for one file and append for another. Defend both choices. What would go wrong if you swapped them?

5.3 Apply (5 pts): The school clinic wants a program that keeps (a) the list of learners with their LRN and section, and (b) a record of every visit, including the time. Which of this week's classes would you use for each, and would each file overwrite or append? Why?

5.4 Reflect (5 pts): Compare your program last week, which forgot everything when it ended, with this week's. What can it do now that it could not do before? What part was hardest?

✂ CUT ON THURSDAY · DETACH THE THURSDAY STRIP BELOW THIS LINE ✂

THURSDAY · OCT 29 Writing: FileWriter, BufferedWriter, StringWriter

Name: _____
Section: _____ Score: _____ / 15

Answer after Step 10 on page 5.

4.1 Define (5 pts): What is the difference between `new FileWriter("log.txt")` and `new FileWriter("log.txt", true)`?

4.2 Explain (5 pts): In Step 7, two lines were written but `trip_log.txt` showed only one. Why? Which line was lost?

4.3 Connect (5 pts): The Error Guide shows a file that stayed **empty** (0 bytes) with no error. What caused it, and how does try-with-resources prevent it?

Week 15 CodeShare practice additions

Use this appendix with the original module. Original lesson pages and sample listings remain unchanged. Text backups and data remain unchanged in the supporting-files folder.

File exercises retain real disk access and the original file setup. CodeShare classroom file access/persistence is not supported by this update. Verify these exercises in a separate desktop Java folder; do not treat this JSON as classroom-ready.

Run the unchanged sample first. Sample verified records the demonstration only. Make Your task changes, then compare the complete Expected task output. A successful alteration earns Completed. Keep module work, personal work and submitted versions separate. Paper liplets remain teacher assessed.

Step 1 alteration

Sample: java-2-w15-d1-s1 Task: java-2-w15-d1-s1-alter-v1

Create fleet.txt twice in the same run. Immediately after printing Created now, call createNewFile() again and print "Created again? " followed by its boolean result. Start in a clean exercise folder.

Expected task output

```
File name: fleet.txt
Exists before? false
Created now? true
Created again? false
Exists after? true
Is a file? true
Size in bytes: 0
```

Step 2 alteration

Sample: java-2-w15-d1-s2 Task: java-2-w15-d1-s2-alter-v1

After deleting old_log.txt twice, delete trips.txt and print "Delete trips.txt? " plus the result. Then run the existing folder deletion and listing. Print an empty array if folder.list() is null because the folder was deleted. Start with a clean records folder.

Expected task output

```
Folder made? true
Is a folder? true
Inside records: [old_log.txt, trips.txt]
Delete old_log.txt? true
Delete it again? false
Delete trips.txt? true
Delete the folder? true
Left inside: []
```

Step 3 alteration

Sample: java-2-w15-d2-s3 Task: java-2-w15-d2-s3-alter-v1

Keep fleet.txt unchanged. Count newline characters instead of commas. Rename the counter to newlines and its final printed label to "Newline characters in the file: ". The supplied file ends with a newline.

Expected task output

```
First line: Jeepney,NXA 123,Mang Ramon,Monumento-Divisoria
Newline characters in the file: 4
```

Step 4 alteration

Sample: java-2-w15-d2-s4 Task: java-2-w15-d2-s4-alter-v1

Load the unchanged four-line fleet file. Before adding a parsed vehicle to the list, include only Bus objects. The loaded count and printed list must reflect filtering, not edited labels.

Expected task output

```
Loaded 1 vehicles from fleet.txt
BUS 789 | Driver: Aling Rosa | Passengers: 0/50
```

Step 5 alteration

Sample: java-2-w15-d3-s5 Task: java-2-w15-d3-s5-alter-v1

Keep fares.txt unchanged. Read each number, then count and add it only when the fare is at least P15.00. Compute the average from the accepted fares.

Expected task output

```
Fares read: 3
Total collected: P63.0
Average fare: P21.00
```

Step 6 alteration

Sample: java-2-w15-d3-s6 Task: java-2-w15-d3-s6-alter-v1

Change countVehicles so it counts only nonblank lines beginning with "Tricycle,". Use the same String, file and empty-String inputs.

Expected task output

```
From a String: 1
From fleet.txt: 1
From an empty String: 0
```

Step 7 alteration

Sample: java-2-w15-d4-s7 Task: java-2-w15-d4-s7-alter-v1

Change only the second FileWriter to append mode. Keep both original write calls. The first trip must survive the second write.

Expected task output

```
--- trip_log.txt ---
NXA 123 left Monumento
TRC 456 left Bagong Silang
```

Step 8 alteration

Sample: java-2-w15-d4-s8 Task: java-2-w15-d4-s8-alter-v1

Begin with the original Step 7 log containing the Tricycle line. Change Step 8 from append mode to overwrite mode. Keep its two write calls; the old Tricycle line must disappear.

Expected task output

```
--- trip_log.txt ---
BUS 789 left Caloocan
NYB 321 left Sangandaan
```

Step 9 alteration

Sample: java-2-w15-d4-s9 Task: java-2-w15-d4-s9-alter-v1

Load the original four vehicles. Replace the extra Tricycle with new Bus("BUS 654", "Mang Lito", 30). Save and reload exactly as before; the final object must have capacity 30.

Expected task output

```
Loaded from fleet.txt: 4
Saved to fleet_backup.txt: 5
Reloaded: 5
BUS 654 | Driver: Mang Lito | Passengers: 0/30
```

Step 10 alteration

Sample: java-2-w15-d4-s10 Task: java-2-w15-d4-s10-alter-v1

Use the original Step 9 backup with five vehicles. Build and save a report containing only Bus records. Skip other lines before increasing count; keep numbering consecutive.

Expected task output

```
DAILY FLEET REPORT
1. BUS 789 (Bus) - Aling Rosa
Total vehicles: 1
report.txt size: 67 bytes
```

Before classroom use

Upload validation is not runtime verification. File exercises retain real disk access and the original file setup. CodeShare classroom file access/persistence is not supported by this update. Verify these exercises in a separate desktop Java folder; do not treat this JSON as classroom-ready.